```
#include <iostream>
#include <type_traits>
#include <string>

template<typename T>
constexpr bool is_basic_type() {
  return
    std::is_arithmetic<T>::value ||    // int, float, double, char, bool, signed, unsigned, long...
    std::is_same<T, std::string>::value; // string
}

using namespace std;
int main() {
   std::cout << is_basic_type<int>() << std::endl;
   std::cout << is_basic_type<long>() << std::endl;
   std::cout << is_basic_type<unsigned int>() << std::endl;
   std::cout << is_basic_type<double>() << std::endl;
   std::cout << is_basic_type<char>() << std::endl;
   std::cout << is_basic_type<std::string>() << std::endl;

   std::cout << is_basic_type<node<int>>() << std::endl;
}
```

```
1
1
1
1
1
1

0
```

```
template<typename D>
void STACK<D>::view(bool v){
   long int index = size();
   if(!index) return;
   if(v) std::cout << "SP->";
   std::cout << "[";
   for(int i = 0; i < index; i++){
    if constexpr (
      std::is_arithmetic<D>::value ||   // int, float, double, char, bool, signed, unsigned, long...
      std::is_same<D, std::string>::value  // string
    )
    { std::cout<<S.value(i);        // string, int, float, double, char, bool, signed, unsigned, long...
    } else { std::cout<<S.value(i).K; }   // node<T>
    if(i < index-1) std::cout << ", ";
   }
   std::cout << "]";
}
```

Your compiler is not using C++17	if constexpr is a C++17 feature Without it → syntax error
Compiler does not understand if constexpr	It treats it as invalid if So else becomes unmatched → error
Both branches get compiled	Even the invalid one: S.value(i).K for int / string ❌
int and string have no .K	So compilation fails

🔧 Fix

g++ Stack.cpp -std=c++17 -o Stack

```
template<typename D>
void QUEUE<D>::view(bool v){
   long int index = size();
   if(!index) return;
   if(v) std::cout << "Front->";
   std::cout << "[";
   for(int i=0; i<index; i++){
      std::cout << Q.value(i); if(i<(index-1)) std::cout<<", ";
   }
   std::cout << "]";
   if(v) std::cout << "<-Rear";
}


template<typename D>
void STACK<D>::view(bool v){
   long int index = size();
   if(!index) return;
   if(v) std::cout << "SP->";
   std::cout << "[";
   for(int i=0; i<index; i++){
      std::cout << S.value(i); if(i<(index-1)) std::cout<<", ";
   }
   std::cout << "]";
}


template<typename D>
std::ostream& operator<<(std::ostream& out, const node<D>& n){
   return out << n.K;
}
```